

## study.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <time.h>
5  #include <ctype.h>
6
7  #define MAX_STUDENT_ID 8      // 2309174 要加1位 \0
8  #define MAX_CHAPTER_CODE 9    // 要考虑最大章节的情况，这里先认为最大单节不超过99 所以这个为例 17.12.11 至少8个字符，要加1位 \0
9  #define MAX_CHAPTER_NAME 100 // 先定义为100
10 #define MAX_COURSE_CODE 7     // CST103 要加1位 \0
11 #define MAX_COURSE_NAME 100   // 先定义为100
12 #define MAX_COURSE_NUMBER 5   // 先定义最大课程数为1，即只学习 CST103
13 #define MAX_CHAPTER_NUMBER 500 // 先定义最大章节数为500，即每课有最多500个章节
14 #define BUFFER_SIZE 1024      // 定义每次读取的知识点文本文件的数据缓存块大小
15 #define MAX_UNIT_FILE 13      // 05.05.01.txt + \0
16 #define MAX_UNIT_NAME 100     // 最大的知识单元名称，先定义为100
17 #define MAX_UNIT_NUMBER 5000
18
19 // 定义课程结构
20 struct Course
21 {
22     char courseCode[MAX_COURSE_CODE];
23     char courseName[MAX_COURSE_NAME];
24 };
25
26 // 定义章节结构
27 struct Chapter
28 {
29     char chapterCode[MAX_CHAPTER_CODE];
30     char chapterName[MAX_CHAPTER_NAME];
31     char courseCode[MAX_COURSE_CODE]; // 记录这些章节是属于什么课程的，这样这个程序可以扩展到其它课程的学习，如数学，马来语等。
32 };
33
34 // 定义知识单元结构
35 struct Unit
36 {
37     char unitFile[MAX_UNIT_FILE];
38     char unitName[MAX_UNIT_NAME];
```

```

39     char chapterCode[MAX_CHAPTER_CODE]; // 知识单元属于哪个章节
40 };
41
42 // 定义学习记录结构
43 struct Study
44 {
45     char studentID[MAX_STUDENT_ID]; // 2309174
46     char courseCode[MAX_COURSE_CODE]; // CST103 等
47     char chapterCode[MAX_CHAPTER_CODE]; // 09 等
48     char unitFile[MAX_UNIT_FILE]; // 05.01.01.txt
49     time_t startTime; // 用时间戳来记录，方便格式化输出各种格式的时间
50     time_t endTime; // 用时间戳来记录，方便格式化输出各种格式的时间
51     time_t totalTime; // 总用时，秒
52 };
53
54 /** 定义函数原型 */
55 char *s_gets(char *st, int n); // 从键盘输入获取字符串
56
57 /* 1. 显示主菜单*/
58 void help(void);
59 int menu(void); // 获取菜单命令的输入
60 int test(void);
61 int show_courses(void); // 显示所有课程
62 int show_chapters(char *course_code); // 根据课程代码，显示章节内容
63 int show_units(char *chapter_code); // 根据章节代码，显示知识单元内容
64 int show_study_records(char *student_id);
65 int choose_units(char *student_id, char *chapter_code); // 根据章节代码，显示知识单元内容，用户输入序号，学习相应的知识点
66 int learn(char *unitFile); // 学习知识单元，参数为知识单元的文件名
67 struct Chapter find_chapter(char *chapter_code);
68 struct Unit find_unit(char *unit_file);
69 void append_study(struct Study study);
70 void update_study(int pos, struct Study study);
71 void save_log(struct Study study);
72 struct Study find_study(char *student_id, char *course_code, char *chapter_code, char *unit_file, int *pos);
73
74 int main(void)
75 {
76     // 程序开始的时候，就先显示菜单
77     menu();
78
79     // 如果选择 q 就退出，为了防止窗口一闪而过，加个getchar();

```

```

80     return 0;
81 }
82
83 // 显示帮助主菜单
84 void help(void)
85 {
86     printf("===== Command Menu =====\n");
87     printf("| To choose a function, enter its letter label:      |\n");
88     printf("| |\n");
89     printf("| a) Show all courses or add them, MAX 3 courses.      |\n");
90     printf("| b) Show all chapters for <CourseCode> or add them, MAX 500 chapters. |\n");
91     printf("| c) Show all units for <ChapterCode> or add them, MAX 5000 units.  |\n");
92     printf("| d) Enter <StudentId> <ChapterCode> to start learning.  |\n");
93     printf("| e) Show all study records for <StudentId>.            |\n");
94     printf("| q) Quit                                                |\n");
95     printf("===== \n");
96     printf("Enter a letter: ");
97 }
98
99 // 获取主菜单命令的输入
100 int menu(void)
101 {
102     char current_course_code[MAX_COURSE_CODE];
103     char current_chapter_code[MAX_CHAPTER_CODE];
104     char current_student_id[MAX_STUDENT_ID];
105     char command;
106     do
107     {
108         help();
109         command = getchar();
110         fflush(stdin);
111         switch (command)
112         {
113             case 'a':
114                 // 显示所有课程以便 这里调用显示所有课程的函数
115                 show_courses();
116                 puts("");
117                 break;
118             case 'b':
119                 printf("Show all chapters for <CourseCode> or press [Enter] to return menu\nEnter CourseCode: ");
120                 if (fgets(current_course_code, sizeof(current_course_code), stdin) != NULL)

```

```

121     {
122         // 调用函数 show_chapters, 并将输入的字符串作为参数传递
123         size_t len = strlen(current_course_code);
124         if (len > 0 && current_course_code[len - 1] == '\n')
125         {
126             current_course_code[len - 1] = '\0';
127         }
128         if (strlen(current_course_code) > 0)
129             show_chapters(current_course_code);
130     }
131     puts("");
132     break;
133 case 'c':
134     printf("Show all units for <ChapterCode> or press [Enter] to return menu\nEnter ChapterCode: ");
135     if (fgets(current_chapter_code, sizeof(current_chapter_code), stdin) != NULL)
136     {
137         size_t len = strlen(current_chapter_code);
138         if (len > 0 && current_chapter_code[len - 1] == '\n')
139         {
140             current_chapter_code[len - 1] = '\0';
141         }
142         if (strlen(current_chapter_code) > 0)
143             show_units(current_chapter_code);
144     }
145     puts("");
146     break;
147 case 'd':
148     puts("Enter <StudentID> <ChapterCode> to start learning, press [Enter] to return menu");
149     scanf("%s %s", &current_student_id, &current_chapter_code);
150     fflush(stdin);
151     if (strlen(current_student_id) > 0 && strlen(current_chapter_code) > 0)
152     {
153         choose_units(current_student_id, current_chapter_code);
154     }
155     /* printf("Enter <ChapterCode> to start learning or press [Enter] to return menu\nEnter ChapterCode: ");
156     if (fgets(current_chapter_code, sizeof(current_chapter_code), stdin) != NULL)
157     {
158         size_t len = strlen(current_chapter_code);
159         if (len > 0 && current_chapter_code[len - 1] == '\n')
160         {
161             current_chapter_code[len - 1] = '\0';

```

```

162         }
163         if (strlen(current_chapter_code) > 0)
164         {
165             choose_units(current_chapter_code);
166         }
167     } */
168
169     puts("");
170     break;
171 case 'e':
172     printf("Show all study records for <StudentId> or press [Enter] to return menu\nEnter StudentId: ");
173     if (fgets(current_student_id, sizeof(current_student_id), stdin) != NULL)
174     {
175         size_t len = strlen(current_student_id);
176         if (len > 0 && current_student_id[len - 1] == '\n')
177         {
178             current_student_id[len - 1] = '\0';
179         }
180         if (strlen(current_student_id) > 0)
181             show_study_records(current_student_id);
182     }
183     puts("");
184     break;
185 default:
186     break;
187 }
188 } while (command != 'q'); // 直到按 q 退出
189 }
190
191 // 显示所有课程内容功能（函数），如果还没有课程，则输入，按 [Enter] 返回
192 int show_courses(void)
193 {
194     puts("");
195
196     // 1. 先定义课程数组，用来存放所有课程
197     struct Course myCourses[MAX_COURSE_NUMBER];
198
199     // 1. 打开文件：定义课程指针，并打开课程文件
200     FILE *pCourse;
201     if ((pCourse = fopen("courses.dat", "a+b")) == NULL)
202     {

```

```
203     puts("The courses.dat file not found.");
204     exit(1);
205 }
206
207 // 读取课程内容并显示
208 rewind(pCourse); // 指针定位到开始处
209 int count = 0;
210 int size = sizeof(struct Course);
211 while (count < MAX_COURSE_NUMBER && fread(&myCourses[count], size, 1, pCourse) == 1)
212 {
213     if (count == 0)
214         printf("Current contents of courses.dat:\n\n");
215     printf("%s: %s\n", myCourses[count].courseCode, myCourses[count].courseName);
216     count++;
217 }
218 puts("");
219
220 // 读取完后，如果超出了最大课程数，则返回，后面不用再录入了
221 int filecount = count;
222 if (count == MAX_COURSE_NUMBER)
223 {
224     puts("The courses.dat file is full.");
225     return 1;
226 }
227
228 // 如果没有超出课程数，则继续录入
229 printf("Continue add course or press [Enter] to stop and show all courses\nEnter CourseCode: ");
230 while (count < MAX_COURSE_NUMBER && s_gets(myCourses[count].courseCode, MAX_COURSE_CODE) != NULL && myCourses[count].courseCode[0]
!= '\0')
231 {
232     printf("Enter CourseName: ");
233     s_gets(myCourses[count].courseName, MAX_COURSE_NAME);
234     count++;
235     if (count < MAX_COURSE_NUMBER) // 如果还没有达到最大课程数，则继续输入
236         printf("Continue add course or press [Enter] to stop and show all courses\nEnter CourseCode: ");
237 }
238 puts("");
239
240 // 输入完课程后，再显示刚才新增的所有课程，因为 count 一直在增长，所以只要从count之后显示即可
241 int index;
242 if (count > 0)
```

```

243 {
244     printf("Current contents of courses.dat:\n\n");
245     for (index = 0; index < count; index++)
246         printf("%s: %s\n", myCourses[index].courseCode, myCourses[index].courseName);
247
248     fwrite(&myCourses[filecount], size, count - filecount, pCourse);
249 }
250 else // 没有录入课程, 直接返回
251     puts("The courses.dat file is empty");
252
253 // 一定要关闭文件指针
254 fclose(pCourse);
255 }
256
257 // 显示指定课程的所有章节, 如果还没有章节, 则输入, 按 [Enter] 返回
258 int show_chapters(char *course_code)
259 {
260     puts("");
261
262     // 先判断课程二进制数据中, 是否存在 course_code 的课程, 如果不存在则提示, 并返回1
263
264     // 先定义章节数组, 用来存放所有章节
265     struct Chapter chapterList[MAX_CHAPTER_NUMBER];
266
267     // 定义章节指针, 并打开章节文件
268     FILE *pChapter;
269     if ((pChapter = fopen("chapters.dat", "a+b")) == NULL)
270     {
271         puts("The chapters.dat file not found.");
272         exit(1);
273     }
274
275     // 读取章节内容并显示
276     rewind(pChapter);
277     int count = 0;
278     int size = sizeof(struct Chapter);
279     while (count < MAX_CHAPTER_NUMBER && fread(&chapterList[count], size, 1, pChapter) == 1)
280     {
281         if (count == 0)
282             printf("Current %s contents of chapters.dat:\n\n", course_code);
283         if (strcmp(chapterList[count].courseCode, course_code) == 0) // 如果是参数指定的课程代码才显示

```

```

284     {
285         printf("%-5s: %s\n", chapterList[count].chapterCode, chapterList[count].chapterName);
286     }
287     count++;
288 }
289 puts("");
290
291 // 读取完后, 如果超出了最大章节数, 则返回, 后面不用再录入了
292 int filecount = count;
293 if (count == MAX_CHAPTER_NUMBER)
294 {
295     puts("The chapters.dat file is full.");
296     return 1;
297 }
298
299 // 如果没有超出章节数, 则继续录入
300 printf("Continue add chapter or press [Enter] to stop and show all chapters\nEnter ChapterCode: ");
301 while (count < MAX_CHAPTER_NUMBER && s_gets(chapterList[count].chapterCode, MAX_CHAPTER_CODE) != NULL && chapterList[count]
.chapterCode[0] != '\0')
302 {
303     strcpy(chapterList[count].courseCode, course_code);
304     printf("Enter ChapterName: ");
305     s_gets(chapterList[count++].chapterName, MAX_CHAPTER_NAME);
306     if (count < MAX_CHAPTER_NUMBER)
307         printf("Continue add chapter or press [Enter] to stop and show all chapters\nEnter ChapterCode: ");
308 }
309 puts("");
310
311 // 输入完章节后, 再显示刚才新增的所有章节, 因为 count 一直在增长,
312 int index;
313 if (count > 0)
314 {
315     printf("Current %s contents of chapters.dat:\n\n", course_code);
316     for (index = 0; index < count; index++)
317     {
318         if (strcmp(chapterList[index].courseCode, course_code) == 0) // 如果是参数指定的课程代码才显示
319             printf("%-5s: %s\n", chapterList[index].chapterCode, chapterList[index].chapterName);
320     }
321
322     fwrite(&chapterList[filecount], size, count - filecount, pChapter);
323 }

```



```

324     else // 没有录入章节，直接返回
325         puts("The chapters.dat file is empty");
326
327     // 一定要关闭打开的文件指针
328     fclose(pChapter);
329 }
330
331 struct Chapter find_chapter(char *chapter_code)
332 {
333     struct Chapter myChapter;
334     struct Chapter chapterList[MAX_CHAPTER_NUMBER];
335     FILE *pChapter;
336     if ((pChapter = fopen("chapters.dat", "r")) == NULL)
337     {
338         return myChapter;
339     }
340
341     // 读取章节内容并显示
342     rewind(pChapter);
343     int count = 0;
344     int size = sizeof(struct Chapter);
345     while (count < MAX_CHAPTER_NUMBER && fread(&chapterList[count], size, 1, pChapter) == 1)
346     {
347         if (strcmp(chapterList[count].chapterCode, chapter_code) == 0)
348         {
349             return chapterList[count];
350         }
351         count++;
352     }
353     return myChapter;
354 }
355
356 // 显示指定课程的所有章节，如果还没有章节，则输入，按 [Enter] 返回
357 int show_units(char *chapter_code)
358 {
359     puts("");
360
361     // 先判断章节二进制数据中，是否存在 chapter_code 的章节，如果不存在则提示，并返回1
362
363     // 先定义章节数组，用来存放所有章节
364     struct Unit unitList[MAX_UNIT_NUMBER];

```

```

365
366 // 定义章节指针，并打开章节文件
367 FILE *pUnit;
368 if ((pUnit = fopen("units.dat", "a+b")) == NULL)
369 {
370     puts("The units.dat file not found.");
371     exit(1);
372 }
373
374 // 读取章节内容并显示
375 rewind(pUnit);
376 int count = 0;
377 int size = sizeof(struct Unit);
378 while (count < MAX_UNIT_NUMBER && fread(&unitList[count], size, 1, pUnit) == 1)
379 {
380     if (count == 0)
381         printf("Current %s contents of units.dat:\n\n", chapter_code);
382     if (strcmp(unitList[count].chapterCode, chapter_code) == 0) // 如果是参数指定的课程代码才显示
383     {
384         printf("%-12s: %s\n", unitList[count].unitFile, unitList[count].unitName);
385     }
386     count++;
387 }
388 puts("");
389
390 // 读取完后，如果超出了最大章节数，则返回，后面不用再录入了
391 int filecount = count;
392 if (count == MAX_UNIT_NUMBER)
393 {
394     puts("The units.dat file is full.");
395     return 1;
396 }
397
398 // 如果没有超出章节数，则继续录入
399 printf("Continue add unit or press [Enter] to stop and show all units\nEnter UnitFile: ");
400 while (count < MAX_UNIT_NUMBER && s_gets(unitList[count].unitFile, MAX_UNIT_FILE) != NULL && unitList[count].unitFile[0] != '\0')
401 {
402     if (strstr(unitList[count].unitFile, ".txt") == NULL)
403     {
404         strcat(unitList[count].unitFile, ".txt");
405     }

```

```

406 // strcpy(unitList[count].courseCode, chapter_code);
407 strcpy(unitList[count].chapterCode, chapter_code);
408 printf("Enter UnitName: ");
409 s_gets(unitList[count++].unitName, MAX_UNIT_NAME);
410 if (count < MAX_UNIT_NUMBER)
411     printf("Continue add unit or press [Enter] to stop and show all units\nEnter UnitFile: ");
412 }
413 puts("");
414
415 // 输入完章节后, 再显示刚才新增的所有章节, 因为 count 一直在增长,
416 int index;
417 if (count > 0)
418 {
419     printf("Current %s contents of units.dat:\n\n", chapter_code);
420     for (index = 0; index < count; index++)
421     {
422         if (strcmp(unitList[index].chapterCode, chapter_code) == 0) // 如果是参数指定的课程代码才显示
423             printf("%-12s: %s\n", unitList[index].unitFile, unitList[index].unitName);
424     }
425
426     fwrite(&unitList[filecount], size, count - filecount, pUnit);
427 }
428 else // 没有录入章节, 直接返回
429     puts("The units.dat file is empty");
430
431 // 一定要关闭打开的文件指针
432 fclose(pUnit);
433 }
434
435 struct Unit find_unit(char *unit_file)
436 {
437     // struct Unit myUnit;
438     struct Unit unitList[MAX_UNIT_NUMBER];
439     FILE *pUnit;
440     if ((pUnit = fopen("units.dat", "r")) == NULL)
441     {
442         // return myUnit;
443         printf("The units.dat file not found.\n");
444         exit(1);
445     }
446

```

```

447 // 读取章节内容并显示
448 rewind(pUnit);
449 int count = 0;
450 int size = sizeof(struct Unit);
451 while (count < MAX_CHAPTER_NUMBER && fread(&unitList[count], size, 1, pUnit) == 1)
452 {
453     if (strcmp(unitList[count].unitFile, unit_file) == 0)
454     {
455         return unitList[count];
456     }
457     count++;
458 }
459 // return myUnit;
460 }
461
462 // 根据章节号查该章节下的所有知识点，并让用户选择学习哪个知识点
463 int choose_units(char *student_id, char *chapter_code)
464 {
465     puts("");
466
467     // 先定义一个单元数组，把单元文件读到的数据放这里
468     struct Unit unitList[MAX_UNIT_NUMBER];
469
470     // 先打开文件，如果文件不存在，则提示，并返回1，一般返回非0 表示出错，非正常执行到尾
471     FILE *pUnit;
472     if ((pUnit = fopen("units.dat", "r")) == NULL)
473     {
474         printf("The units.dat file not found.\n");
475         return 1;
476     }
477
478     // 循环读取单元文件的数据，并把它放到单元数组中，判断只有等于 参数指定的章节号 的才显示出来，读取完成后，关闭指针
479     rewind(pUnit);
480     int count = 0;
481     int size = sizeof(struct Unit);
482     while (count < MAX_UNIT_NUMBER && fread(&unitList[count], size, 1, pUnit) == 1)
483     {
484         if (count == 0)
485             printf("Current %s contents of units.dat:\n\n", chapter_code);
486         if (strcmp(unitList[count].chapterCode, chapter_code) == 0) // 如果是参数指定的课程代码才显示
487         {

```

```
488         printf("%2d) %-12s: %s\n", count + 1, unitList[count].unitFile, unitList[count].unitName);
489     }
490     count++;
491 }
492 fclose(pUnit);
493 puts("");
494
495 // 一直等待用户输入序号,
496 printf("Enter number to start learning, or [Non-numeric] to return: ");
497
498 // 获取输入的字符, 判断是不是数字, 如果不是就返回
499 char num;
500 num = getchar();
501 fflush(stdin);
502 if (num == '\n' || !isdigit(num))
503     return 2;
504
505 // 如果是数字, 就找到这个序号对应的单元, 开始学习, 这时要记录开始时间
506 time_t start_time = time(NULL);
507 int number = num - '0';
508 char unit_file[MAX_UNIT_FILE];
509 strcpy(unit_file, unitList[number - 1].unitFile);
510
511 // 显示知识点的内容, 并一直提示用户, 只有按 e 键才结束学习, 否则一直显示让学生学习。
512 int is_end = learn(unit_file) == 0;
513
514
515 if (is_end) // 如果按了结束标志 e
516 {
517     // 取得当前时间, 并用当前时间 - 开始时间, 就是这次学习的总时长
518     time_t end_time = time(NULL);
519     time_t total_time = end_time - start_time;
520
521     // 这个时候要开始写入学习记录了, 因为学习记录中要有课程号, 所以要先通过章节号找到课程号
522     char course_code[MAX_COURSE_CODE];
523     strcpy(course_code, find_chapter(chapter_code).courseCode);
524
525     // 先从学习记录中找到这个知识点的记录, 看是否已经有学习过, 有就找出来更新, 没有就添加新的学习记录
526     int pos = -1;
527     struct Study myStudy = find_study(student_id, course_code, chapter_code, unit_file, &pos);
528 }
```

```

529     if (pos == -1)// 如果没有找到学习记录, 把数据填充到本次学习结构体中,
530     {
531         strcpy(myStudy.studentID, student_id);
532         strcpy(myStudy.courseCode, course_code);
533         strcpy(myStudy.chapterCode, chapter_code);
534         strcpy(myStudy.unitFile, unit_file);
535         myStudy.startTime = start_time;
536         myStudy.endTime = end_time;
537         myStudy.totalTime = total_time;
538
539         // 并添加到二进制学习记录文件中, 把这次学习的 study 结构内容, 作为参数直接传给 append_study
540         append_study(myStudy);
541     }
542     else // 如果已经有学习过这个知识点, 就把这个知识点原来的学习时间 + 本次学习的时间, 然后更新学习记录
543     {
544         myStudy.totalTime += total_time;
545
546         // 然后更新这个学习记录, 更新时要指定更新哪条数据, 所以这里要指定, 上面 find_study 找到的序号, 并把这次学习的 study 结构内容, 作
为参数直接传给 update_study
547         update_study(pos, myStudy);
548     }
549
550     // 保存完二进制学习记录后, 还要保存本次的学习日志, 把这次学习的 study 结构内容, 作为参数直接传进去, 在 save_log 中就可以取到数据了
551     save_log(myStudy);
552
553     // 保存完后, 继续显示这个章节的其它知识点, 让学生选择是否继续学习其它知识点
554     choose_units(student_id, chapter_code);
555 }
556
557 return 0;
558 }
559
560 void save_log(struct Study study)
561 {
562     FILE *pStudy;
563     if ((pStudy = fopen("study.log", "a+")) == NULL)
564     {
565         printf("The study.log file not found.\n");
566         exit(1);
567     }
568     fprintf(pStudy, "%s,%s,%s,%s,%d,%d,%d\n", study.studentID, study.courseCode, study.chapterCode, study.unitFile, study.startTime,

```

```

study.endTime, study.totalTime);
569     fclose(pStudy);
570 }
571
572 void append_study(struct Study study)
573 {
574     FILE *pStudy;
575     if ((pStudy = fopen("study.dat", "a+b")) == NULL)
576     {
577         printf("The study.dat file not found.\n");
578         exit(1);
579     }
580
581     int size = sizeof(struct Study);
582
583     fwrite(&study, size, 1, pStudy);
584
585     // free(studyList);
586
587     fclose(pStudy);
588 }
589
590 void update_study(int pos, struct Study study)
591 {
592     FILE *pStudy;
593     if ((pStudy = fopen("study.dat", "rb+")) == NULL)
594     {
595         printf("The study.dat file not found.\n");
596         exit(1);
597     }
598
599     int size = sizeof(struct Study);
600     fseek(pStudy, pos * size, SEEK_SET);
601
602     fwrite(&study, size, 1, pStudy);
603
604     fclose(pStudy);
605 }
606
607 int show_study_records(char *student_id)
608 {

```

```
609 puts("");
610
611 // 先判断课程二进制数据中，是否存在 course_code 的课程，如果不存在则提示，并返回1
612
613 // 定义章节指针，并打开章节文件
614 FILE *pStudy;
615 if ((pStudy = fopen("study.dat", "rb")) == NULL)
616 {
617     puts("The study.dat file not found.");
618     exit(1);
619 }
620
621 int size = sizeof(struct Study);
622 long fileSize, recordCount;
623 fseek(pStudy, 0, SEEK_END);
624 // 获取文件大小
625 fileSize = ftell(pStudy);
626 // 计算记录总数
627 recordCount = fileSize / size;
628 // 先定义章节数组，用来存放所有章节
629 struct Study *studyList = (struct Study *)malloc(recordCount * size);
630
631 // 读取章节内容并显示
632 rewind(pStudy);
633 int count = 0;
634 while (count < recordCount && fread(&studyList[count], size, 1, pStudy) == 1)
635 {
636     if (count == 0)
637     {
638         printf("StudentId %s study records in study.dat:\n\n", student_id);
639         printf("%-15s%-15s%-15s%-15s%-15s\n", "StudentID", "CourseCode", "ChapterCode", "UnitFile", "TotalTime");
640     }
641     if (strcmp(studyList[count].studentID, student_id) == 0)
642     {
643         printf("%-15s%-15s%-15s%-15s%-15d\n",
644             studyList[count].studentID, studyList[count].courseCode, studyList[count].chapterCode, studyList[count].unitFile,
645             studyList[count].totalTime);
646     }
647     count++;
648 }
649 puts("");
```



```

649     fflush(stdin);
650     free(studyList);
651
652     // 一定要关闭打开的文件指针
653     fclose(pStudy);
654     return 0;
655 }
656
657 /**
658  * 根据传入的参数，查找知识点学习记录，返回 Study 的结构体，
659  */
660 struct Study find_study(char *student_id, char *course_code, char *chapter_code, char *unit_file, int *pos)
661 {
662     struct Study myStudy = {};
663
664     FILE *pStudy; // 定义章节指针，并打开章节文件
665     if ((pStudy = fopen("study.dat", "a+b")) == NULL)
666     {
667         puts("The study.dat file not found.");
668         return myStudy;
669     }
670
671     int size = sizeof(struct Study);
672     long fileSize, recordCount;
673     fseek(pStudy, 0, SEEK_END);
674     fileSize = ftell(pStudy); // 获取文件大小
675     recordCount = fileSize / size; // 计算记录总数
676     struct Study *studyList = (struct Study *)malloc(recordCount * size); // 先定义章节数组，用来存放所有章节
677
678     rewind(pStudy); // 读取章节内容并显示
679     int count = 0;
680     while (count < recordCount && fread(&studyList[count], size, 1, pStudy) == 1)
681     {
682         if (strcmp(studyList[count].studentID, student_id) == 0 && strcmp(studyList[count].courseCode, course_code) == 0 &&
683             strcmp(studyList[count].chapterCode, chapter_code) == 0 && strcmp(studyList[count].unitFile, unit_file) == 0)
684         {
685             *pos = count;
686             return studyList[count];
687         }
688         count++;
689     }

```

```

689     return myStudy;
690 }
691
692 int learn(char *unitFile)
693 {
694     puts("");
695
696     FILE *pUnit;
697     if ((pUnit = fopen(unitFile, "r")) == NULL)
698     {
699         printf("The %s file not found.\n", unitFile);
700         return 1;
701     }
702     char buffer[BUFFER_SIZE]; // 定义每次读取的数据块, 比 fgetc 一个字符一个字符读取效率更高,
703     printf("[START OF %s]\n", unitFile);
704     while (fgets(buffer, BUFFER_SIZE, pUnit) != NULL)
705     {
706         printf("%s", buffer);
707     }
708     // 关闭打开的文件指针
709     fclose(pUnit);
710
711     printf("\n[END OF %s]\n\n", unitFile);
712
713     printf("Press [e] to end the study: ");
714     char end = getchar();
715     fflush(stdin);
716     if (end == 'e')
717     {
718         puts("end");
719         return 0;
720     }
721     learn(unitFile);
722 }
723
724 // 获取输入字符串功能
725 char *s_gets(char *st, int n)
726 {
727     fflush(stdin);
728     char *ret_val;
729     char *find;

```

```
730     ret_val = fgets(st, n, stdin);
731     if (ret_val)
732     {
733         find = strchr(st, '\n');
734         if (find)
735             *find = '\0';
736         else
737             while (getchar() != '\n')
738                 continue;
739     }
740     return ret_val;
741 }
742
743 int test(void)
744 {
745     time_t start = time(NULL);
746     printf("starting timestamp: %d, time: %s\n", start, ctime(&start));
747
748     // sleep(10); // 假设学习了10秒
749
750     time_t end = time(NULL);
751     printf("complete timestamp: %d, time: %s\n", end, ctime(&end));
752
753     time_t study_total_time = end - start;
754     printf("Total study time is: %d seconds\n", study_total_time);
755 }
756
```